

SVTT-T11: Dự án API NodeJS (dành cho sinh viên)

1. Giới thiệu mục tiêu

Dự án yêu cầu bạn xây dựng hệ thống quản lý công việc (Task Management) mà không sử dụng framework. Mục tiêu cốt lõi là giúp bạn làm chủ:

- HTTP Methods:** Phân biệt và sử dụng đúng GET, POST, PUT, DELETE.
- Data Persistence:** Cách lưu trữ dữ liệu bền vững vào tệp tin JSON bằng module `fs`.
- URL Parsing:** Tự xử lý việc bóc tách ID từ địa chỉ URL (ví dụ: `/tasks/123`).
- Stream & Buffer:** Cách gom dữ liệu thô từ client gửi lên thành một đối tượng JSON hoàn chỉnh.

2. Mô tả Task Model (Cấu trúc dữ liệu)

Dựa trên mã nguồn triển khai, mỗi đối tượng Task trong hệ thống sẽ tuân thủ cấu trúc sau để đảm bảo tính nhất quán khi lưu trữ vào file `tasks.json`:

Trường (Field)	Kiểu dữ liệu	Mô tả
id	<code>String</code>	Định danh duy nhất, được tạo bằng <code>Date.now().toString()</code> .
title	<code>String</code>	Tiêu đề công việc (Dữ liệu bắt buộc từ Client).
description	<code>String</code>	Chi tiết công việc (Mặc định là chuỗi rỗng <code>""</code> nếu không cung cấp).

Trường (Field)	Kiểu dữ liệu	Mô tả
completed	Boolean	Trạng thái hoàn thành (Mặc định là <code>false</code> khi tạo mới).

Ví dụ một bản ghi trong `tasks.json`:

```
{
  "id": "1712835600000",
  "title": "Học Node.js thuần",
  "description": "Thực hành đọc ghi file JSON",
  "completed": false
}
```

3. Cấu trúc hệ thống 4 tầng

Để mã nguồn sạch sẽ và dễ bảo trì, dự án được chia theo mô hình sau:

- **Tầng 1. Server (`server.js`):** Tiếp nhận kết nối và chuyển giao toàn bộ quyền xử lý cho Router.
- **Tầng 2. Router (`router.js`):** Bộ não điều hướng. Phân tích phương thức HTTP, trích xuất ID từ URL và tích hợp logic `parseBody` để gom các mảnh dữ liệu thô (Buffer) thành JSON.
- **Tầng 3. Controller (`controllers/`):** Người điều phối. Nhận dữ liệu từ Router, gọi chức năng ở Service và thực hiện phản hồi (Response) kèm mã trạng thái phù hợp.
- **Tầng 4. Service (`services/`):** Người thực thi. Trực tiếp thao tác đọc/ghi file `tasks.json`, xử lý logic thêm/xóa/sửa dựa trên dữ liệu Model.

4. Danh sách API chi tiết

Chức năng	Phương thức	Đường dẫn	Mô tả nhiệm vụ
Lấy danh sách	GET	<code>/tasks</code>	Trả về toàn bộ mảng tasks từ file JSON.
Xem chi tiết	GET	<code>/tasks/:id</code>	Tìm task theo ID trong file JSON.

Chức năng	Phương thức	Đường dẫn	Mô tả nhiệm vụ
Tạo mới	POST	/tasks	Khởi tạo ID, mặc định <code>completed: false</code> và lưu vào file.
Cập nhật	PUT	/tasks/:id	Ghi đè các trường thông tin dựa trên ID (giữ nguyên ID cũ).
Xóa	DELETE	/tasks/:id	Lọc bỏ task khỏi mảng và ghi lại vào file.

5. Các Module Node.js sử dụng

Dự án này sử dụng các thư viện tích hợp sẵn (Core Modules):

- **http:** Khởi tạo server và quản lý Request/Response.
- **fs/promises:** Thao tác đọc/ghi file JSON bất đồng bộ.
- **path:** Xử lý đường dẫn file hệ thống.

6. Luồng dữ liệu hệ thống (Flow)

(Sơ đồ quy trình xử lý yêu cầu từ Client đến Server)

- **Bước 1:** Client gửi Request.
- **Bước 2:** Server nhận Request và chuyển cho Router.
- **Bước 3:** Router phân tích URL & Method.
 - Nếu có Body (POST/PUT): Tích hợp `parseBody` để gom Buffer.
 - Nếu không Body (GET/DELETE): Trích xuất ID trực tiếp từ URL.
- **Bước 4:** Controller tiếp nhận dữ liệu và điều phối.
- **Bước 5:** Service thực hiện đọc/ghi file `tasks.json`.
- **Bước 6:** Server trả kết quả JSON & Status Code về Client.

7. Tiêu chí hoàn thành (Checklist)

- **Không dùng thư viện ngoài:** Không sử dụng `npm install`.
- **Lưu trữ bền vững:** Dữ liệu không bị mất khi restart server nhờ lưu vào file.
- **Xử lý bất đồng bộ:** Phải sử dụng `async/await` cho các hàm xử lý file.
- **Phản hồi chuẩn:** Trả về đúng mã trạng thái (200 OK, 201 Created, 404 Not Found).
- **Tổ chức mã nguồn:** Logic đọc ghi file (readDB/writeDB) phải nằm ở tầng Service.

Phiên bản #2

Được tạo 11 tháng 4 2026 04:19:39 bởi TienPVd

Được cập nhật 11 tháng 4 2026 04:24:55 bởi TienPVd